Mathematical techniques

1. Induction and course-of-value induction
2. Countable and uncountable sets

Languages and automata

1. Regular expression
2. Deterministic finite automaton, error state
3. Nondeterministic finite automaton, nondeterministic finite automaton with ε-moves
4. You should know Kleene's theorem, and in particular, how to algorithmically convert a regular expression to a DFA. You don't need to know how to convert a DFA to a regular expression.
5. You should be able to prove that a language isn't regular. (The Pumping Lemma wasn't covered and isn't needed.)
6. Equivalence of automata
7. Automata for complement
8. Minimization
9. Context free languages (not pushdown automata)
10. Ambiguity of context free grammars.

Complexity

1. Complexity of algorithms vs complexity of problems
2. Lower and upper bounds
3. O, Ω, θ notation
4. Polynomial and exponential complexity
5. $\mathcal{NP}$ problems, the two definitions, the question of whether $\mathcal{P} = \mathcal{NP}$. (You're not required to know the answer.)
6. $\mathcal{NP}$-completeness and SAT

Turing machines

1. Execute Turing machines
2. Design Turing machines for simple tasks
3. Fancy Turing machines (extended alphabet, 2 tape etc.) can be simulated by an ordinary Turing machine (without affecting polynomial time complexity)
4. Church's thesis: any function on words that can be computed algorithmically can be computed by a Turing machine

Decidability

1. Problem, decision problem (a problem whose answer is yes/no)
2. Decidable and semidecidable problems
3. Primitive recursive functions: you need to be able to show that a function is primitive recursive by implementing it in Primitive Java
4. Ackermann function: it is example of a function that is not primitive recursive. (Remembering the exact definition is not required.)
5. The Halting Theorem: whether a given program halts on given inputs is undecidable. (Knowing the proof is not required.)
6. Rice's Theorem: any semantic property of code that sometimes hold and sometimes fails to hold is undecidable. (Knowing the proof is helpful to understand other problems.)
7. If you're asked to show a problem is decidable, write a program to decide it. Or at least sketch how you would write a program.
8. If you're asked to show a problem is undecidable, you might reduce the halting problem to it, or you might appeal to Rice's theorem.

Note that at each point you should know practical aspects, e.g. uses of regular and context free languages, consequences of non-computability, etc.